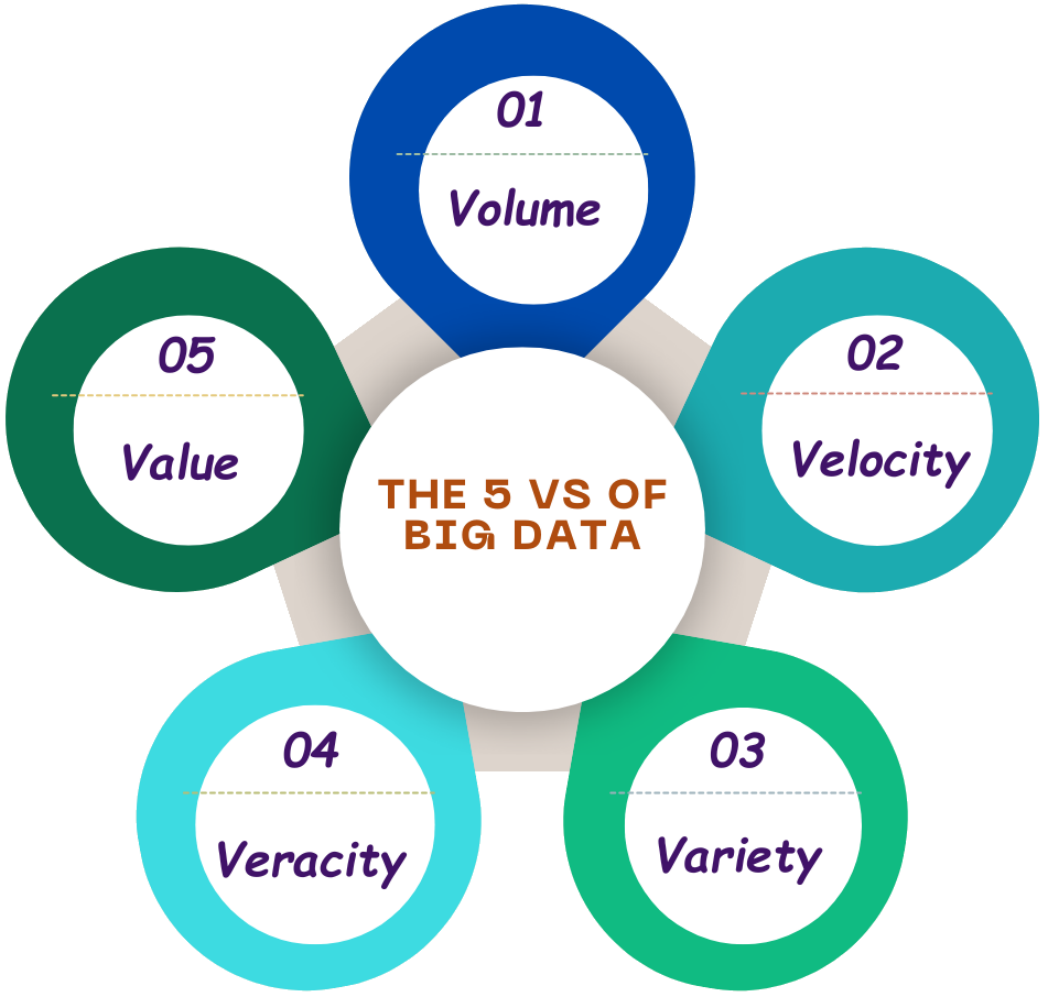
HADOOP

Md Redwan Hasan

Software Engineering, SUST

What is Big Data?

Big Data is data that is hard to store, process, and analyze using traditional systems.



- Big Data is not only “large size.”
- The data may arrive quickly, come in many formats, and contain messy or incomplete values.
- The main question is: can we extract useful value from it?

The 5 Vs describe the challenge

Key idea: Hadoop became famous mainly for storing and processing very large batch data.

Why Hadoop was needed

The old solution was “buy a bigger server.” Hadoop popularized “add more machines.”

Scale up: one bigger server

- Simple at small scale
- Becomes expensive quickly
- Storage and CPU have limits
- Failure can stop everything

Hadoop thinking

distributed storage +
distributed processing

Scale out: many machines

- Add more low-cost machines
- Split data into blocks
- Process blocks in parallel
- Replicas protect against failure

Key idea: Hadoop is built around scale-out: many computers cooperating as one cluster.

A little history of Hadoop

Hadoop grew from search-engine scale problems and open-source distributed computing.

2003

Google File System paper inspires distributed storage.

2004

Google MapReduce paper inspires parallel data processing.

2006

Hadoop starts from open-source search work.

2008+

Apache Hadoop becomes popular in Big Data.

2010s

Hive, Pig, Spark, HBase and other tools grow around it.

Today

Concepts still matter in data lakes and clusters.

Key idea: History matters because Hadoop was designed for web-scale distributed storage and processing.

Hadoop in one simple model

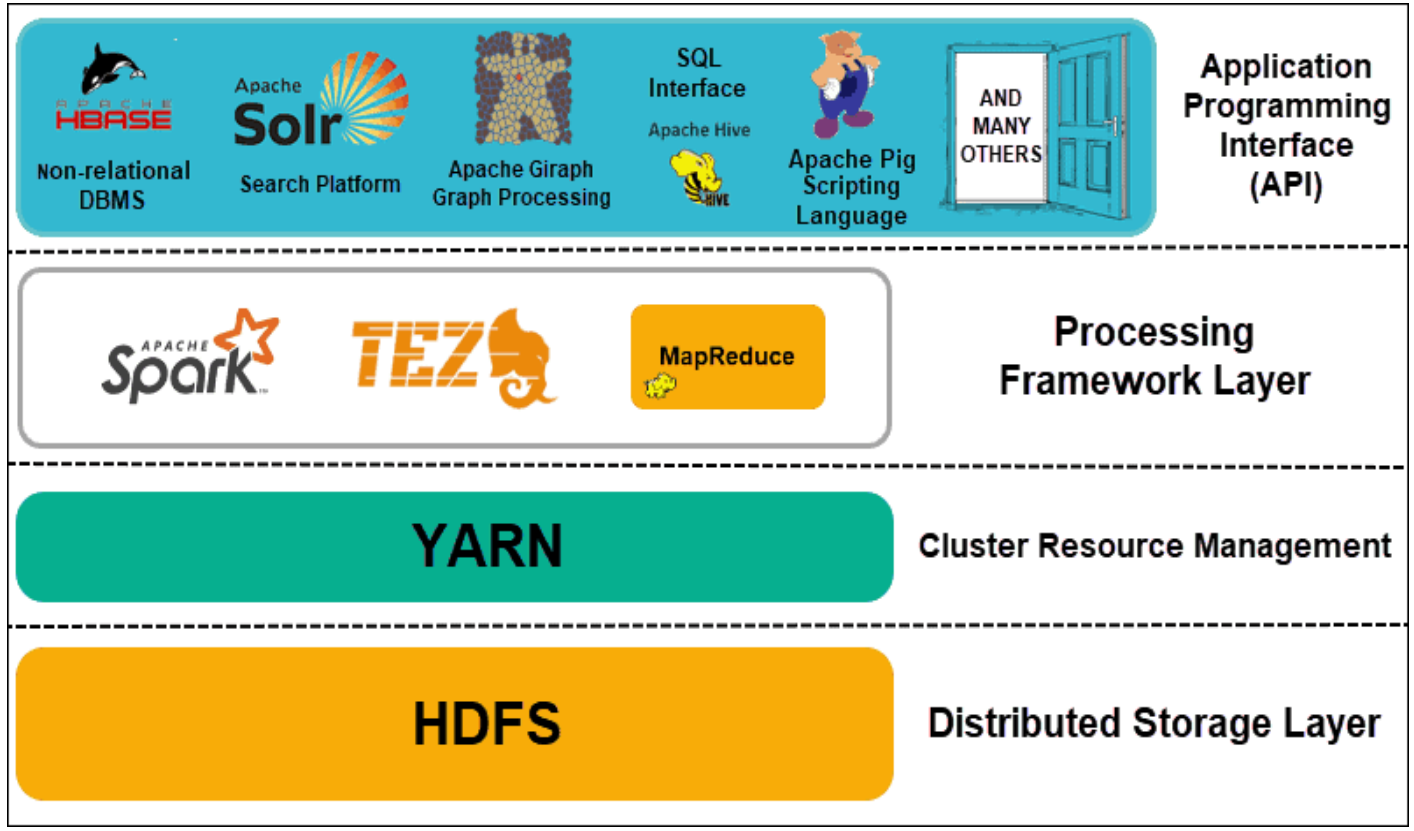
Three core ideas: store the data, manage the cluster, process the data.

- HDFS stores files across many machines.
- YARN decides where work can run.
- Processing engines run tasks on the cluster.

Key idea: If you remember only one thing: HDFS stores, YARN manages, processing engines compute.

High-level Hadoop ecosystem layers

Use this image to explain the big picture before going into details.



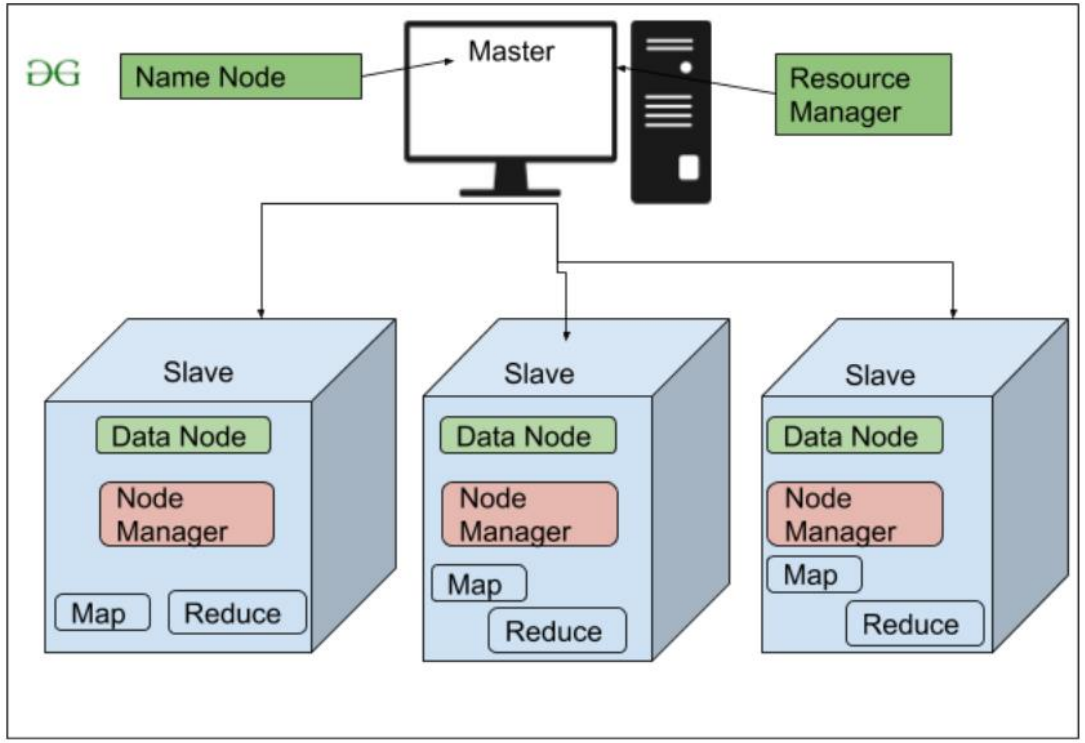
- Bottom: HDFS stores the data.
- Middle: YARN manages cluster resources.
- Processing: MapReduce, Spark, Tez run computation.
- Top: APIs and tools make Hadoop easier to use.

Do not memorize every logo. First understand which layer each tool belongs to.

Key idea: Architecture becomes simple when you read it from bottom to top.

Master-worker cluster idea

A Hadoop cluster has manager services and many worker machines.



- Master-side services coordinate the cluster.
- Worker nodes store data and run tasks.
- In a real cluster, roles can be separated across machines for reliability.
- For learning, a single laptop can run a pseudo-distributed setup.

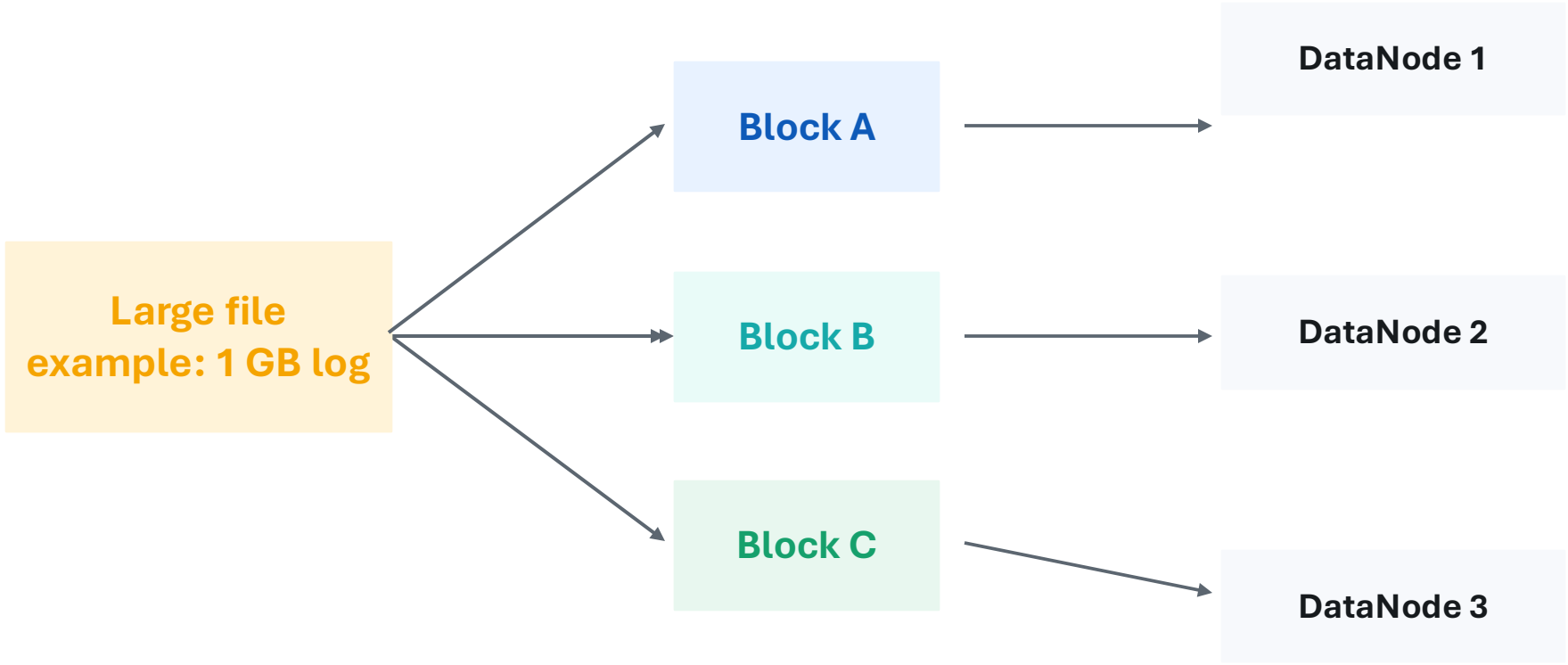
Beginner translation

Manager tells workers what to do; workers hold data and execute tasks.

Key idea: Hadoop is a coordinated group of machines, not just one program.

HDFS: the storage layer

HDFS means Hadoop Distributed File System.



- HDFS splits a file into blocks.
- Blocks are copied to multiple machines.
- NameNode stores metadata; DataNodes store actual blocks.

Key idea: HDFS stores one logical file as many replicated blocks across worker nodes.

HDFS terms you must know

These terms appear again and again in Hadoop theory and practical commands.

Block

A large chunk of a file stored in HDFS. A file is split into blocks.

Replication

Extra copies of a block stored on different DataNodes for safety.

NameNode

The master for HDFS metadata: file names, permissions, block locations.

DataNode

A worker that stores actual data blocks on disk.

fsimage

A checkpoint file of the file-system metadata.

edits

A log of recent metadata changes after the checkpoint.

Secondary NameNode

Helps merge fsimage and edits; it is not a hot backup NameNode.

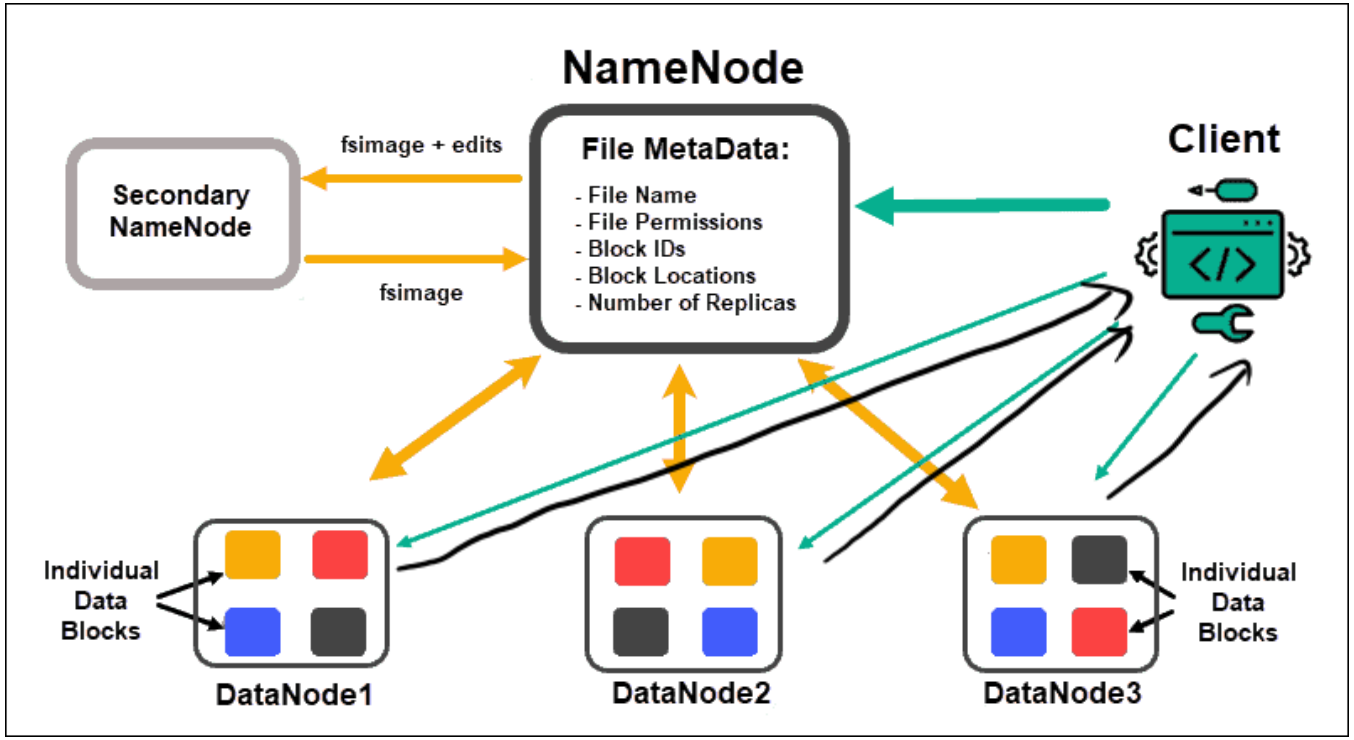
Rack awareness

Replication strategy that places copies across racks to reduce data loss risk.

Key idea: Learn these terms before watching HDFS diagrams; the diagrams will make more sense.

HDFS components: NameNode and DataNodes

This attached image shows where metadata and data blocks live.

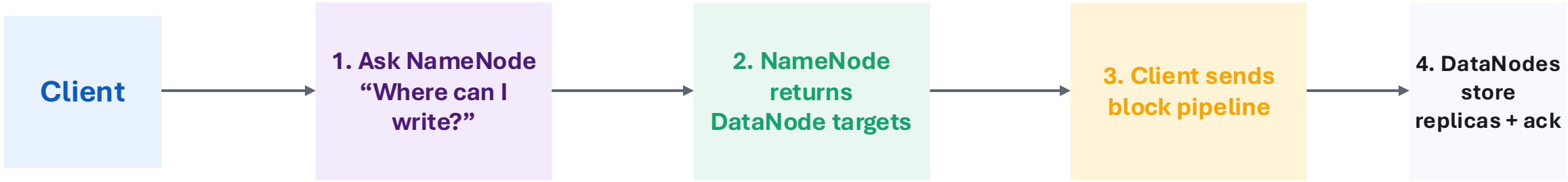


- Client asks the NameNode for block locations.
- NameNode replies with metadata and locations.
- Client reads/writes block data directly with DataNodes.
- DataNodes send heartbeats and block reports to the NameNode.

Key idea: NameNode knows where data is; DataNodes hold the actual data.

HDFS write flow — corrected arrows

Use this simple process flow when explaining how a file is saved into HDFS.

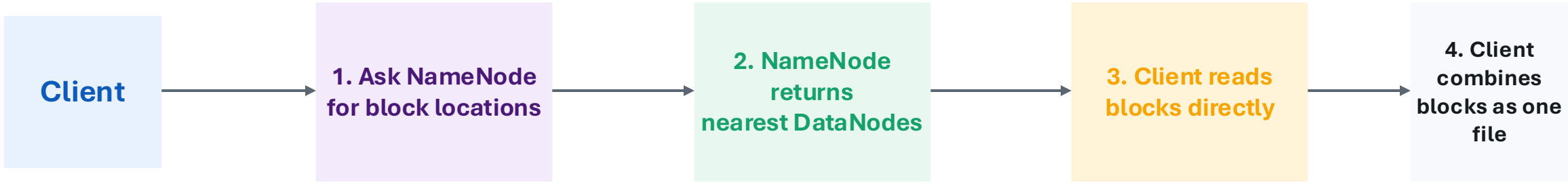


- The NameNode does not store the file content.
- The client writes data blocks to DataNodes.
- DataNodes acknowledge after replicas are written.

Key idea: Write flow direction: Client → NameNode for metadata → DataNodes for actual data.

HDFS read flow — corrected arrows

Reading is also metadata first, then data blocks from DataNodes.

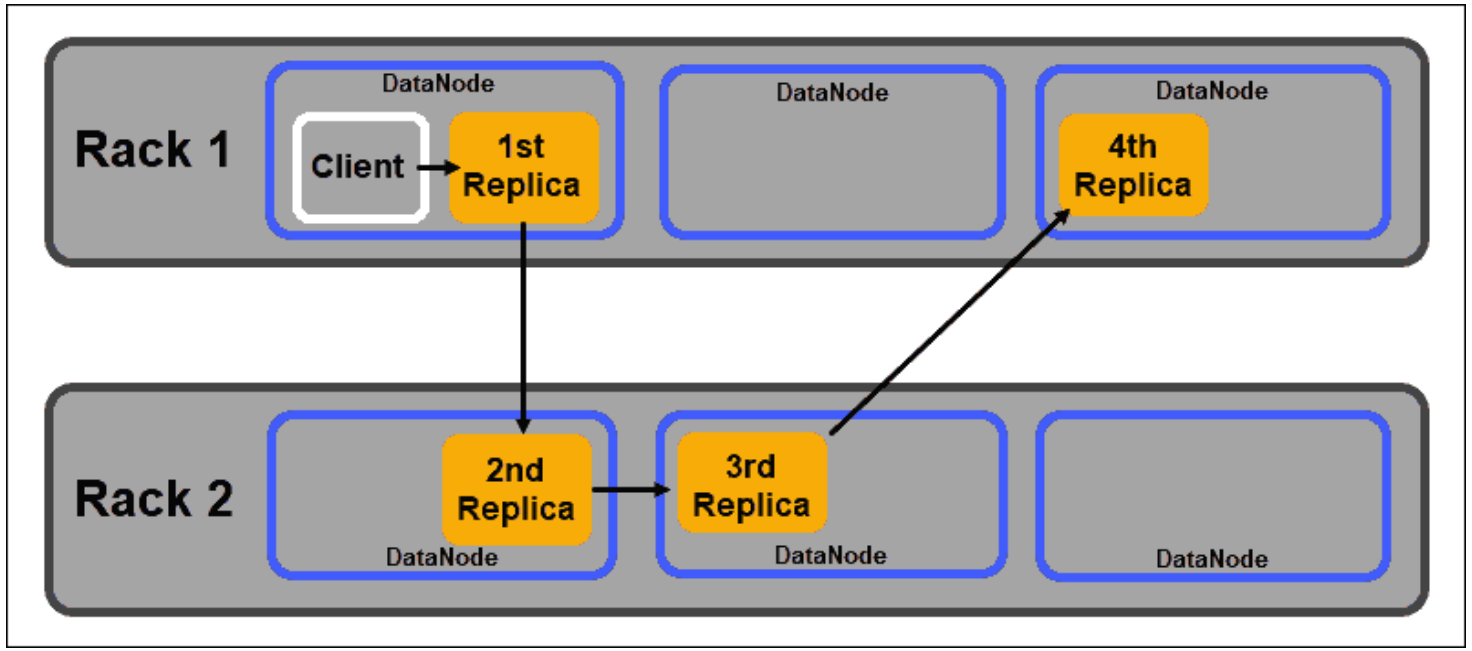


- Hadoop tries to read from a nearby replica.
- If one DataNode fails, the client can read another replica.
- This is how HDFS gets reliability and throughput.

Key idea: Read flow direction: Client → NameNode for location → DataNodes for blocks.

Rack awareness and replication

HDFS does not randomly place all replicas in the same place.

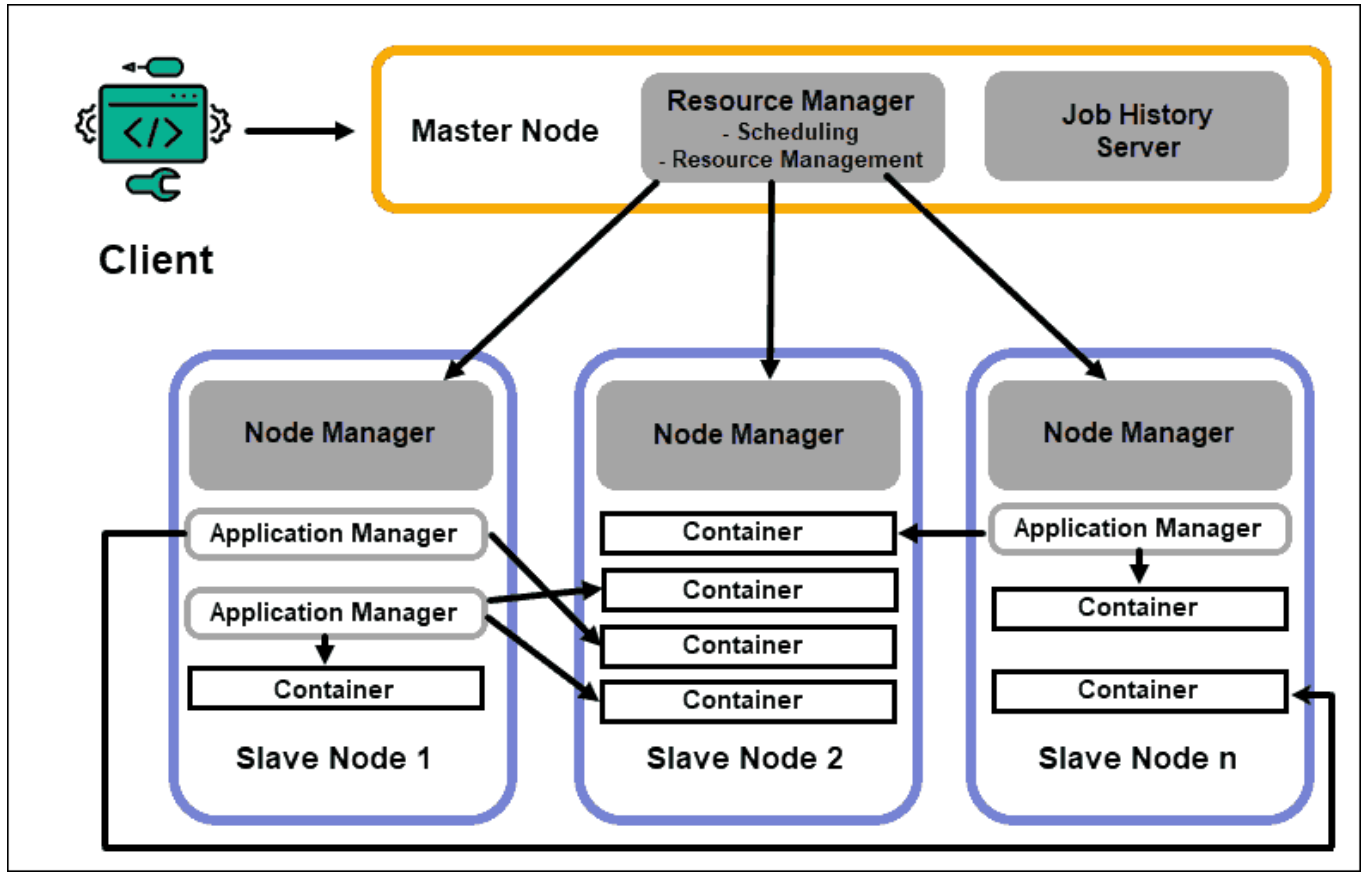


- A rack is a group of machines connected through the same network switch.
- If all replicas stay in one rack, one rack failure can lose availability.
- HDFS spreads replicas to balance reliability and network cost.

Key idea: Replication makes HDFS fault tolerant; rack awareness makes replication smarter.

YARN: the cluster resource manager

YARN decides who gets CPU and memory on the cluster.



- ResourceManager: the master scheduler for resources.
- NodeManager: agent on each worker node.
- ApplicationMaster: manager for one application/job.
- Container: allocated CPU + memory where a task runs.

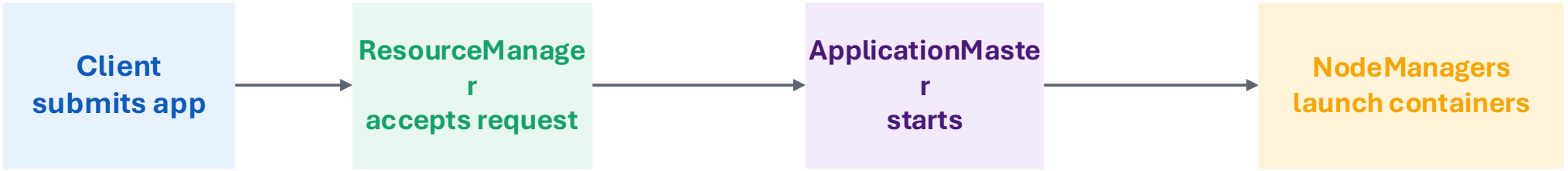
Beginner analogy

YARN is like a traffic controller: it decides where work can safely run.

Key idea: YARN separates resource management from data processing.

YARN terms in one job launch

This corrected flow avoids mixed arrow directions.

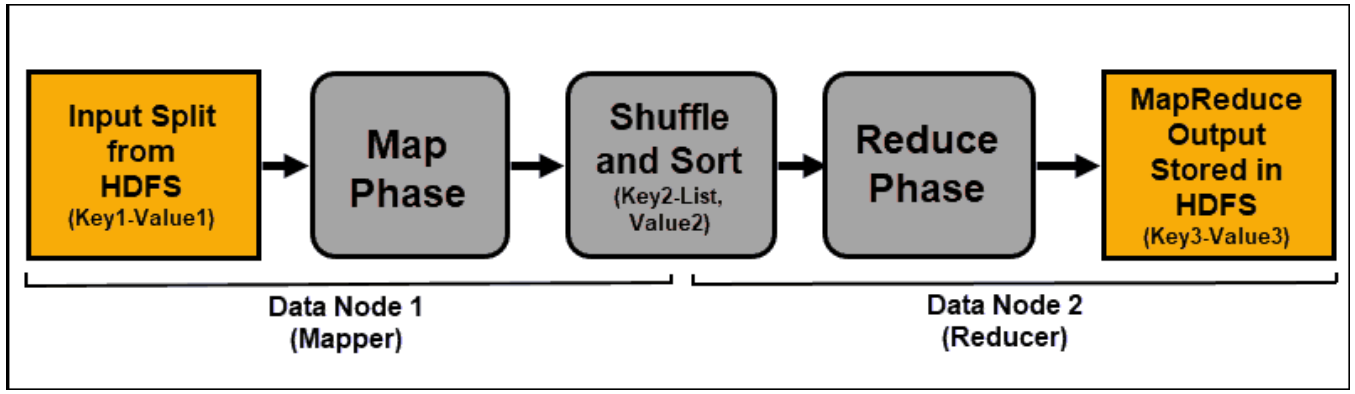


- ResourceManager schedules resources for the whole cluster.
- ApplicationMaster manages one application.
- NodeManager starts containers on its machine.
- Container is where a map or reduce task actually runs.

Key idea: YARN is the reason many different engines can share the same cluster.

MapReduce: the processing idea

MapReduce breaks a big job into small parallel tasks.



- Map phase: read input and create key-value pairs.
- Shuffle and sort: group the same keys together.
- Reduce phase: combine grouped values and write final output.

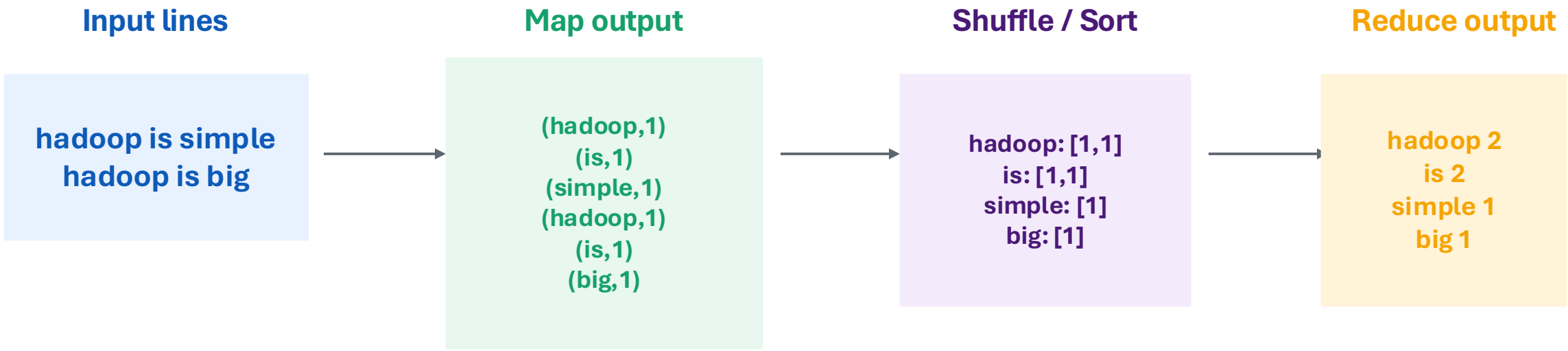
Easy example: Word Count

Input text → words become keys → same words grouped → counts added together

Key idea: MapReduce is mostly about transforming data into key-value pairs and aggregating them.

Word Count example: Map → Shuffle → Reduce

Use this example in the video because it is easy to visualize.



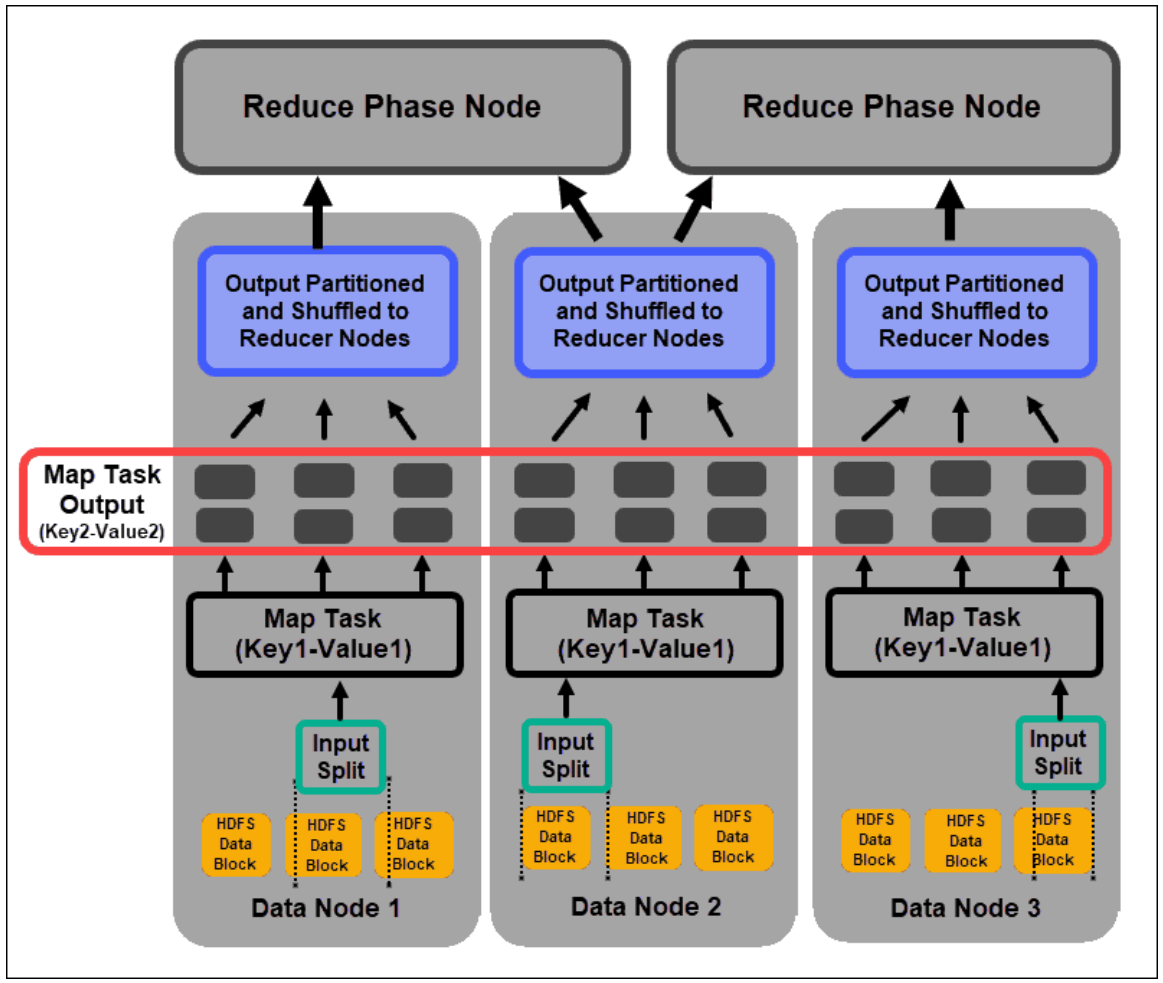
- Mapper does not need to know the final answer.
- Shuffle groups the same keys from all mappers.
- Reducer calculates the final value for each key.

- **Counting and Summation**: Calculating frequencies, such as word counts, log entries, or total sales.
- **Filtering and Searching**: Extracting specific records that match criteria, like finding all users from a specific city.
- **Sorting**: Organizing massive datasets into a specific order across the cluster.
- **Aggregation**: Computing mathematical metrics like averages, minimums, maximums, or standard deviations.
- **Joining**: Combining data from two different datasets based on a common key, similar to an SQL JOIN.

Key idea: The magic is not the counting; the magic is parallel grouping and combining at scale.

Map phase in more detail

A file is split, map tasks run near data, and intermediate key-value output is produced.



- InputSplit is the piece of input assigned to one map task.
- Mapper reads records and emits intermediate key-value pairs.
- Map output is local temporary data first, not final HDFS output.
- Then it is partitioned and prepared for shuffle.

Correct direction

HDFS block/input split → map task → intermediate output → shuffle.

Key idea: The map phase converts raw input into structured key-value pairs.

Shuffle and Sort: the bridge between map and reduce

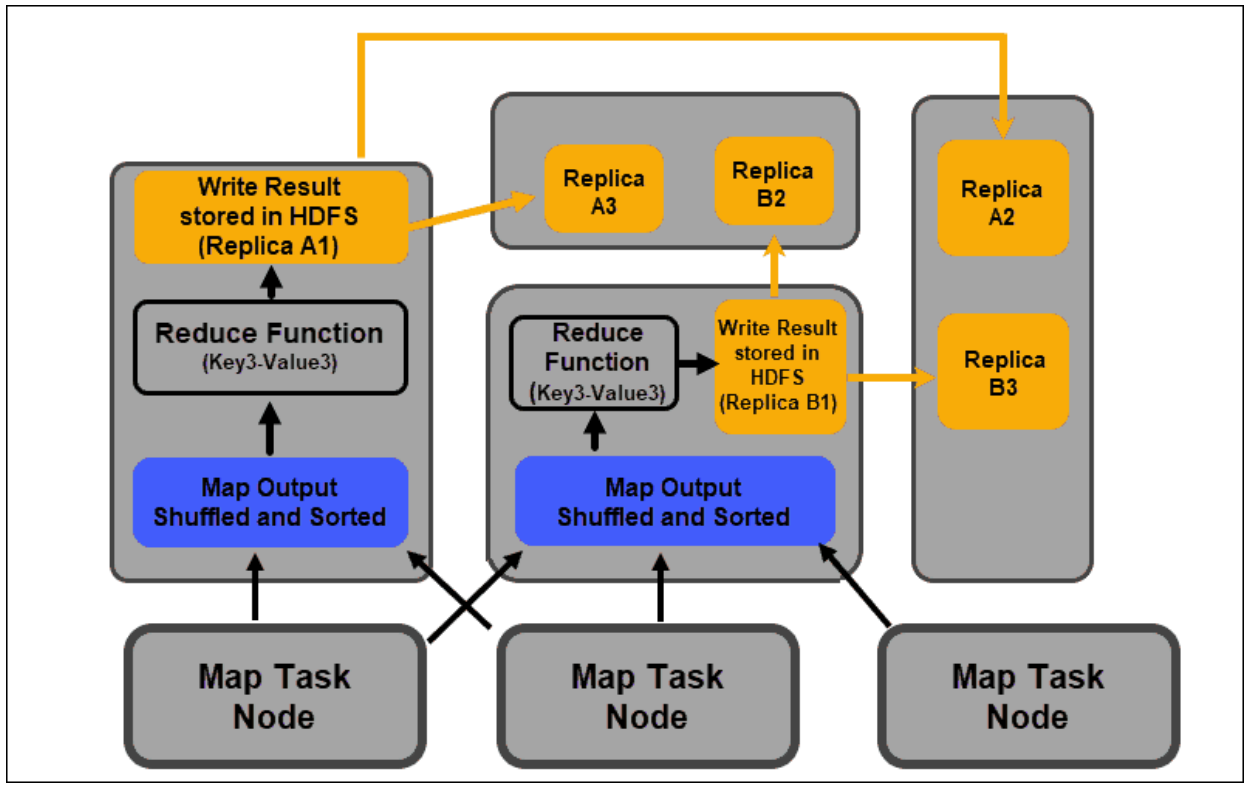
This is the most important MapReduce step to understand.

- Partitioner decides the target reducer for each key.
- Shuffle transfers intermediate data across the network.
- Sort groups the same keys so reducer receives key + list of values.
- This step can be expensive because it involves disk and network I/O.

Key idea: Shuffle and sort connects parallel map work to final grouped reduce work.

Reduce phase and final HDFS output

Reducers combine grouped values and write result files back to HDFS.



- Each reducer receives a group of keys.
- Reducer function combines values for each key.
- Final output is written back to HDFS.
- Output usually appears as part files, for example part-r-00000.
- The number of reducers affects the number of final output files.

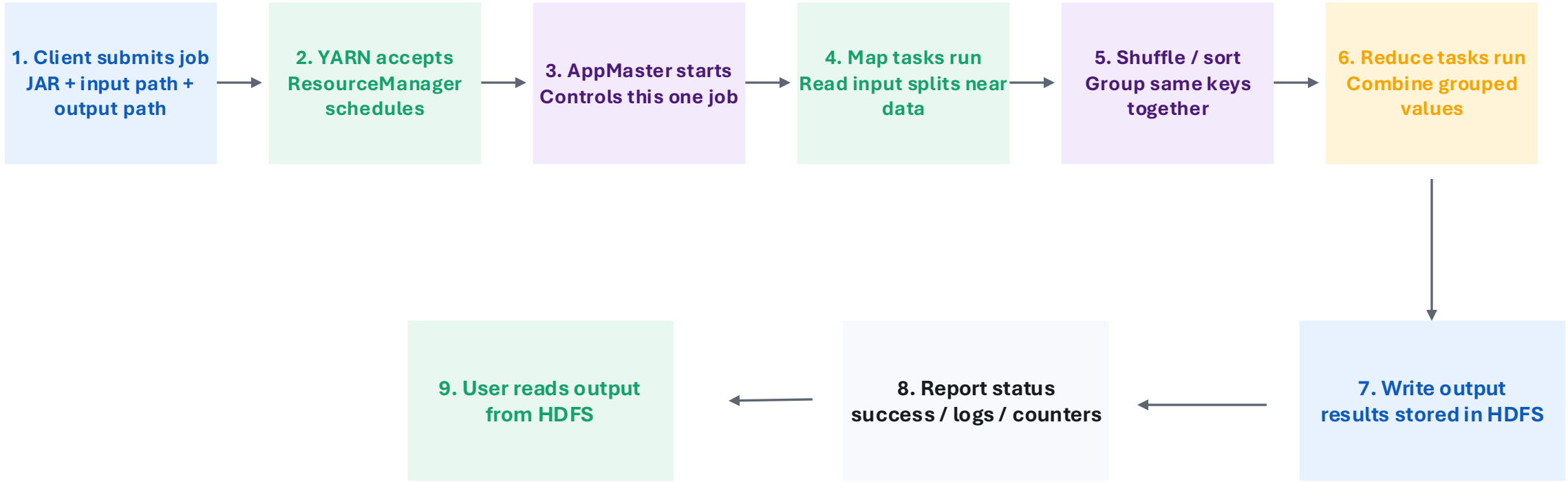
Simple flow

Grouped data → reduce function → final files in HDFS.

Key idea: Reduce turns grouped intermediate data into final results.

How a Hadoop job actually executes

This is the full beginner-friendly process flow with consistent arrows.



Key idea: A job is not one big process; it is many tasks coordinated by YARN and reading/writing HDFS.

Two ideas that make Hadoop powerful

Data locality and fault tolerance are the reasons Hadoop can scale on many machines.

Data locality

- Move computation near the data instead of moving huge data across the network.
- Map tasks often run on the node that already has the input block.
- This saves network bandwidth and improves speed.

Fault tolerance

- Blocks have replicas on multiple DataNodes.
- If a task fails, Hadoop can retry it on another node.
- If a DataNode fails, HDFS can use another replica.

Key idea: Hadoop expects failures and designs around them.

Hadoop frameworks: what each one is for

Frameworks sit around Hadoop to make storage, querying, processing, and management easier.

Hive

SQL-like data warehouse on Hadoop. Good for analysts who know SQL.

Pig

Data-flow scripting. Useful historically for ETL-style pipelines.

Spark

Fast general-purpose processing engine that can use HDFS and YARN.

HBase

NoSQL database on top of HDFS for random read/write access.

Sqoop

Imports/exports data between relational databases and Hadoop. Mostly legacy now.

Flume / Kafka

Ingest streaming event/log data into big data systems.

Oozie / Airflow

Schedule and manage data workflows.

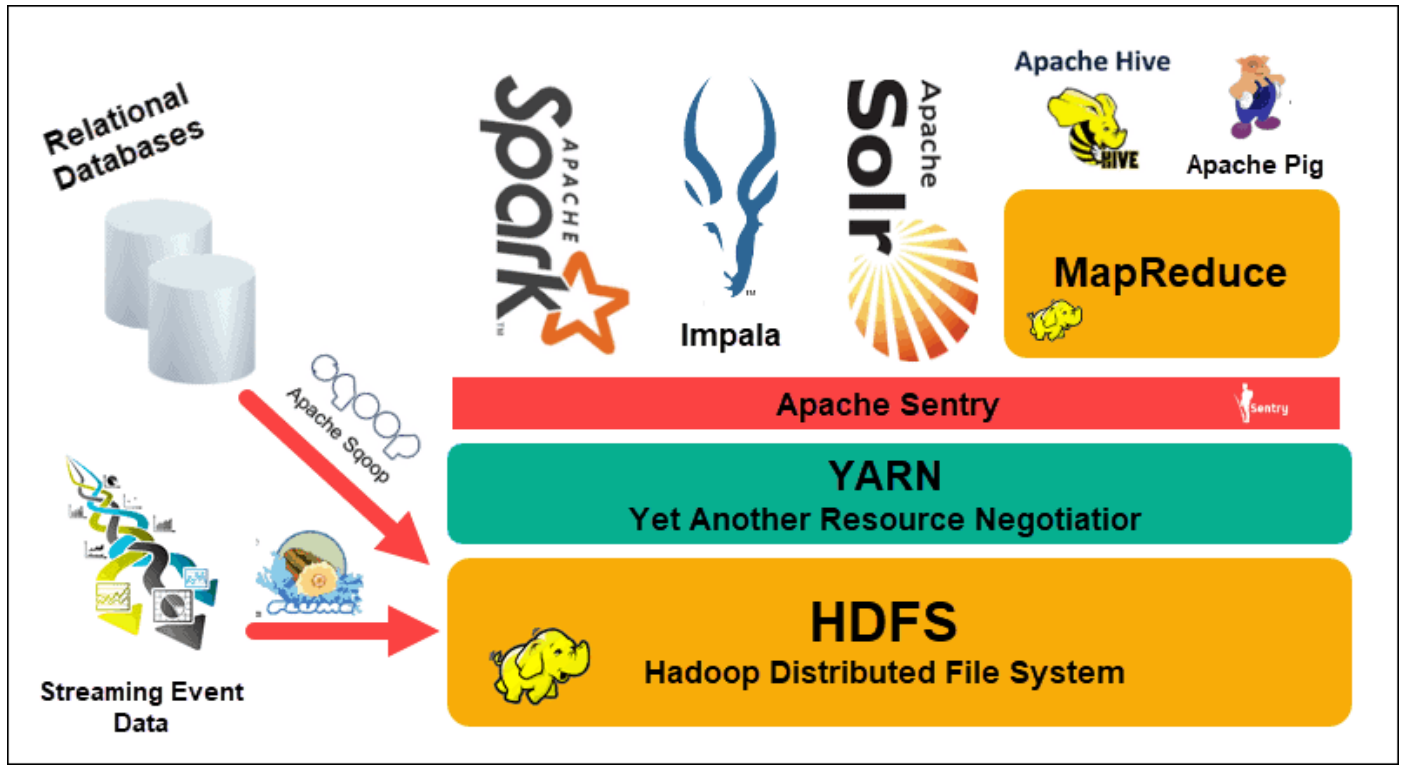
Ranger / Sentry

Security: permissions, authorization, and governance.

Key idea: Frameworks make Hadoop usable for SQL, ETL, streaming, NoSQL, security, and workflow tasks.

Hadoop ecosystem projects architecture

This attached image shows how many tools connect around HDFS and YARN.



- HDFS is the base storage layer.
- YARN manages compute resources.
- MapReduce, Spark, Hive, Pig, Impala and Solr provide ways to process/query data.
- Security and governance tools control access.
- Data may enter from relational databases, streaming systems, or files.

Important note:

Some ecosystem tools are optional. You do not need all of them to start practicing Hadoop.

Key idea: Start with HDFS + YARN + one processing engine before studying every ecosystem tool.

Practical setup path for beginners

After theory, start with pseudo-distributed Hadoop on one machine.

Step 1	Install Java and Hadoop
Step 2	Configure Hadoop environment variables
Step 3	Edit core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml
Step 4	Format NameNode and start HDFS/YARN daemons
Step 5	Run the built-in WordCount example

Beginner goal

Make HDFS work
then run one MapReduce job
then inspect output

Key idea: Do not start with a large cluster. First learn the flow on one machine.


THANK YOU FOR YOUR ATTENTION
DONT ASK QUESTIONS
makeameme.org